A Machine Learning Decision Layer for Feedback-State Selection in Introductory Programming

Alimzhan Koshikov
School of Software Engineering
Astana IT University
Astana, Kazakhstan
225148@astanait.edu.kz
ORCID: 0009-0003-3234-6487

Shyryn Tutkyshbayeva
School of Software Engineering
Astana IT University
Astana, Kazakhstan
sh.tutkyshbayeva@astanait.edu.kz
ORCID: 0000-0002-1419-2571

***Abstract*—Virtual mentors for novice programmers need a reliable decision layer before they generate feedback. This paper studies that layer as a feedback-state classification problem within SocratesAI, a prototype that combines a Spring Boot backend, a FastAPI inference service, and interaction logging for later evaluation. A public dataset of Python Codeforces submissions is filtered to beginner-level problems and official judge outcomes are mapped to five mentor states: accepted, semantic debug, execution safety, efficiency review, and syntax repair. The main diagnostic benchmark contains 178,414 submissions from 872 problems after beginner-level filtering and class balancing. The source-code model, which uses hashed character n-grams, reaches a macro F1 of 0.426 compared with 0.085 for the majority baseline. After run-result summaries are included, the post-execution macro F1 reaches 0.814. To check whether this result depends on the selected split, we additionally ran a three-split grouped evaluation and a split-first experiment with an uncapped test partition. The model ordering remained the same in both experiments. The contribution is an explicit feedback-state formulation and a deployable decision layer for SocratesAI rather than a claim of classroom learning gains. The results indicate that source code is useful for early estimates, while execution signals are most useful as post-execution refinement evidence.**

***Index Terms*—virtual mentor, programming education, machine learning, automated feedback, Codeforces, SocratesAI**

I. Introduction

Introductory programming is often difficult for beginners because they have to deal with several things at once: syntax, algorithmic reasoning, debugging, and testing. Because of this, student submissions can fail for very different reasons. Some contain syntax errors, others produce unsafe execution, incorrect logic, or solutions that are too inefficient. Judge output helps identify that something went wrong, but it does not always show what kind of feedback the student actually needs. A syntax error, for example, usually requires help with a specific part of the code. A time-limit error is more related to the efficiency of the chosen algorithm. For a wrong answer, it is often more useful to focus on edge cases or on the difference between the expected and actual output.

Many recent systems in programming education use large language models (LLMs) to generate explanations and hints [4]–[7]. This is useful, but there is still an important step that comes before generating the feedback. The system first needs to understand what kind of help is suitable for the student's current state. If this decision is incorrect, even a well-written explanation may focus on the wrong problem and lead the student in the wrong direction.

This paper focuses on this decision layer. Instead of building a complete tutoring system, we study whether a machine learning model can predict an appropriate feedback state from the student's source code and, when available, from the results of code execution. The experiments are implemented within SocratesAI, a prototype that uses a Java Spring Boot backend together with a Python FastAPI ML service. The backend sends the student's submission for feedback-state prediction and then uses the predicted action to guide the feedback generation process. The predicted state and its confidence score are also stored in `interaction_logs` for later analysis.

The paper addresses the following research questions:

- **RQ1.** Can a learned feedback-state model generalize to submissions from beginner-level problems that were not used during training?
- **RQ2.** How much does prediction quality improve when source code is combined with run results such as passed tests, time, and memory?
- **RQ3.** Which feedback states remain unreliable, and how should SocratesAI handle low-confidence predictions?

The contribution of the paper is threefold:

- it defines an operational feedback-state taxonomy that maps Codeforces judge outcomes to mentor decisions before free-form hint generation;
- it evaluates this decision layer on 178414 real Python submissions using a problem-level holdout that keeps unseen programming tasks out of training;
- it connects the trained model to SocratesAI through a confidence-aware inference path and structured interac-

tion logging.

The novelty therefore lies in the feedback-state formulation and system integration, not in proposing a new classifier architecture.

II. Related Work

Automated support for programming exercises covers many feedback mechanisms. Keuning et al. review automated feedback generation for programming exercises, while the more recent surveys by Messer et al. and Kumar et al. show how broad the design space has become, including test-based feedback, repair-oriented feedback, and adaptive explanation systems [1]–[3]. In this area, Wang et al. introduced SARFGEN, a data-driven system that generates repair-oriented feedback for introductory programming tasks using previous student solutions [13]. This work shows that useful programming feedback can be automated. However, the main focus is still on generating or repairing feedback after the system has already identified the problem in the student's submission.

Another line of work emphasizes diagnosing the kind of error a student faces. Becker et al. study compiler-error enhancement for novice programmers, showing the value of improving how low-level failures are presented [11]. Lobanov et al. classify recurring error types in programming-assignment submissions from an online learning platform [12]. This diagnostic perspective is close to the present paper: before producing pedagogical language, the system should estimate which failure mode is most plausible from observable evidence.

This issue becomes even more important with LLM-based assistants. Denny et al. discuss desirable properties of AI teaching assistants in programming education, including reliability and alignment with learning goals [4]. CodeAid reports a real classroom deployment of an LLM-based programming assistant [6], Iris explores an AI-driven tutor for computer science education [7], Liu et al. describe how human feedback was used to improve the CS50 AI assistant [8], and Ahmed et al. study AI-augmented feedback for CS1 [5]. Sheese et al. further show that LLM tools change student help-seeking behaviour [9], while Marwan et al. show the value of adaptive immediate feedback tied to learner state [10]. The present paper differs from those studies by focusing on the intermediate decision layer: it does not claim a full tutoring system or automatic program repair system, but tests whether a compact feedback-state classifier can decide the pedagogical direction before hint generation.

III. Method

A. Feedback-State Formulation

Let

$$D = \{(x_i, y_i, g_i)\}_{i=1}^{n}$$

be a dataset of programming submissions. Here x_i is the observable submission information, y_i is the feedback state, and g_i is the problem identifier. The feature vector is

$$x_i = (c_i, m_i, e_i),$$

TABLE I
Mapping from judge outcomes to feedback states

`Codeforces outcome`	Feedback state
`OK`	Accepted
`WRONG_ANSWER, PARTIAL, FAILED, REJECTED, CHALLENGED, SKIPPED`	Semantic debug
`RUNTIME_ERROR, MEMORY_LIMIT_EXCEEDED, CRASHED`	Execution safety
`TIME_LIMIT_EXCEEDED, IDLENESS_LIMIT_EXCEEDED`	Efficiency review
`COMPILATION_ERROR`	Syntax repair

TABLE II
Feedback-state taxonomy used for model training

Feedback state	Typical mentoring decision
Accepted	Do not interrupt with corrective feedback.
Semantic debug	Ask about logic, boundary cases, or expected versus actual output.
Execution safety	Focus on unsafe operations, indexing, recursion depth, or memory use.
Efficiency review	Guide the student toward complexity and repeated work.
Syntax repair	Localize language-form errors before conceptual feedback.

where c_i is source code, m_i contains problem and static-code metadata, and e_i contains optional run-result signals.

The feedback state is a compact class describing the type of intervention needed by the mentor. In this paper,

$$Y = \{\text{accepted}, \text{semantic debug}, \text{execution safety}, \text{efficiency review}, \text{syntax repair}\}.$$

The states are derived from official judge outcomes. The exact mapping used before filtering and training is shown in Table I. For example, a wrong answer is treated as a semantic-debug need, a time limit failure as efficiency review, a runtime or memory failure as execution safety, and a compilation error as syntax repair. This mapping is not a teacher-written hint. It is an operational target for the decision layer; the mentoring interpretation is summarized in Table II.

A classifier estimates

$$\hat{y}_i = f(\phi(x_i)),$$

where $\phi(\cdot)$ is a feature representation. For the linear models used here, each class has a score

$$s_k(x_i) = w_k^{\top}\phi(x_i) + b_k.$$

The parameters are fitted by regularized logarithmic loss:

$$\min_{W,b} -\sum_{i=1}^{n} \log P(y_i \mid \phi(x_i); W, b) + \lambda \sum_{k} \|w_k\|_2^2.$$

Problems are separated between training and testing:

$$G_{\text{train}} \cap G_{\text{test}} = \varnothing.$$

This split is stricter than a random row split because it tests whether the model works on new tasks, not only on new submissions to already seen tasks.

B. Dataset and Features

The empirical source is a local copy of the Codeforces Python submissions dataset [15]. It was selected because it fixes the programming language family, contains accepted and failed submissions, provides official judge outcomes, and includes run-result fields such as passed tests, time, and memory.

The original dataset contains 690,396 Python submissions. After limiting the data to beginner-level problems with a rating of 1200 or lower and removing verdicts that do not belong to the defined taxonomy, 490,357 submissions from 881 problems remain. At this stage, the dataset is strongly dominated by the two largest classes, accepted and semantic debug. Together, they contain 411,943 submissions, which corresponds to 84.0% of all filtered rows. To reduce this imbalance without removing examples from minority classes, the training script limits each class to at most 50,000 submissions. In practice, this cap affects only the accepted and semantic debug classes. After applying it, the final modeled dataset contains 178,414 submissions from 872 problems, while the share of the two largest classes decreases to 56.0%. This step is used only to control class imbalance during macro-F_1-oriented training and should not be interpreted as the real distribution of student errors in a classroom.

The source code of each submission is represented using hashed character 3–5-grams. This representation works well for program code because even short character sequences can capture useful patterns such as operators, indentation, brackets, delimiters, and identifier structure. At the same time, it does not require exact variable names to be shared between different solutions.

Several static features are also extracted from every submission, these include the number of lines, source-code length, blank lines, comments, average and maximum line length, branches, loops, functions, classes, imports, input calls, print calls, and exception-handling constructs. These features provide additional information about the structure and overall complexity of the submitted program.

The model also receives information about the programming problem itself, including the problem rating, its tags, and the Python version used for the submission. For the code-and-run setting, we additionally include execution-related features such as the number of passed tests, execution time in milliseconds, and memory usage in bytes. Importantly, the verdict itself is not provided to the model as an input feature. Instead, the execution-enriched model uses only derived information from the run results, avoiding direct use of the label from which the proxy mentor state is created.

The main classifier used in this study is Scikit-learn's `SGDClassifier` with logarithmic loss and L_2 regularization [14]. It was selected because the resulting feature vectors are sparse and high-dimensional. A linear model trained with stochastic gradient descent is a practical choice for this setting: it is relatively fast to train, easy to rerun under fixed experimental settings, and can be serialized and used directly by the Python service in SocratesAI.

TABLE III
CONFIGURATION USED FOR THE REPORTED CODEFORCES STUDY

Item	Setting
Split	`GroupShuffleSplit(test_size=0.25, random_state=42)` grouped by `problem_id`.
Row cap	`max_rows_per_class=50000`; only accepted and semantic-debug are truncated.
Code text	First 5000 source characters per submission.
Code features	`HashingVectorizer`, `analyzer=char_wb`, 3–5-grams, 65536 features, `alternate_sign=False`, L_2 normalization.
Tag features	`TfidfVectorizer` with tokenized problem tags and `min_df=2`.
Numeric features	Median imputation plus `StandardScaler(with_mean=False)`.
Categorical features	`OneHotEncoder(handle_unknown=ignore)` for language variant.
Classifier	`SGDClassifier(loss=log_loss, penalty=l2, alpha=1e-5, max_iter=60, tol=None, class_weight=balanced, random_state=42)`.
Software	`scikit-learn 1.7.1`, `pandas 2.3.2`, and `pyarrow 24.0.0`.

At this stage, the goal is to understand how much mentoring signal can already be extracted from code fragments, static metrics, problem context, and execution results before moving to heavier neural models. The main comparison therefore varies the available evidence under one fixed linear classifier family rather than searching exhaustively over model families.

C. Experimental Protocol and Reproducibility

Table III lists the fixed settings used for the main study. The split contains 135752 training submissions from 654 problems and 42662 test submissions from 218 problems, with zero problem overlap. Because the goal is to test generalization to unseen tasks, the evaluation keeps complete problems out of the training partition. The feature extraction and SGD settings were fixed before the comparative reruns and reused across all reported ablations. The main tables report one grouped holdout with `random_state`=42 after class capping, because the primary comparison is macro-F_1-oriented and should keep the rarer mentor states visible. To test robustness beyond that benchmark, a follow-up analysis repeats the grouped evaluation with `random_state` values 42, 52, and 62, compares additional shallow classifier families on a shared 128-component SVD representation of the source-code features, and also reruns the study under a split-first protocol that caps only the training partition while leaving the test partition untouched.

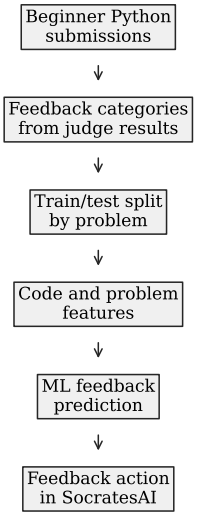


Fig. 1. Pipeline from beginner programming submissions to a feedback-state model used by SocratesAI. Test problems are kept separate from training problems.

TABLE IV
BASELINE AND FEATURE-SET COMPARISON ON A SINGLE HELD-OUT PROBLEM SPLIT (42662 TEST SUBMISSIONS)

Model	**Acc.**	**Bal. acc.**	**Macro F_1**	**W. F_1**
Majority	0.2716	0.2000	0.0854	0.1161
Context metrics	0.3342	0.3483	0.3102	0.3040
Source code	0.4051	0.4248	0.4262	0.4045
Source + context	0.4039	0.4240	0.4121	0.3941
Code + run results	0.7808	0.8207	0.8137	0.7763

IV. EXPERIMENTAL RESULTS

Five models are compared: a majority baseline, a context-metrics model, a source-code model, a source-context model, and a code-and-run model. The reported metrics are accuracy, balanced accuracy, macro F_1, and weighted F_1. Macro F_1 is the main metric because the mentor should not be evaluated only on the largest classes.

Table IV reports the class-balanced diagnostic benchmark. The purpose of this comparison is to isolate which evidence sources improve the decision layer under one fixed linear classifier family, not to claim that SGD is the best possible classifier family.

Baseline comparison. The majority baseline is intentionally weak, but it is still informative: on the 42662-submission test split it collapses every case into semantic debug, classifies only 11589 submissions correctly, and yields macro $F_1 = 0.085$. The context-metrics baseline is stronger, reaching macro $F_1 = 0.310$, yet it still performs very poorly on execution safety ($F_1 = 0.078$). Adding raw source text is the first step that makes every feedback state meaningfully learnable. The source-code model improves macro F_1 by 0.116 over context metrics and by 0.341 over the majority baseline, with the clearest gains on syntax repair ($F_1 = 0.549$) and efficiency review ($F_1 = 0.440$). The source+context variant does not improve over source code alone on this split, so the paper does not claim that simple metadata and static counts consistently add value once character n-grams are already available.

The strongest result is obtained when run results are added. The code-and-run model reaches macro $F_1 = 0.814$ and balanced accuracy 0.821. This model should be read as an evidence upper bound for the moment after execution evidence is available, not as a replacement for the source-only path that SocratesAI can use while the student is still editing. The supported conclusion is therefore narrower than saying that judge output solves mentoring: if SocratesAI has test-run evidence, it should treat that evidence as a central modelling signal, not only as a final grade.

Because the target taxonomy is itself derived from final Codeforces verdicts, a direct rule mapping from the final verdict token to the same labels would be perfect by construction and is therefore not a meaningful ML baseline. The code-and-run result should instead be interpreted more narrowly: it estimates the feedback state from derived run summaries such as passed tests, time, and memory, without seeing the verdict token itself. In SocratesAI, that corresponds to post-execution refinement when the mentor has execution telemetry but still needs to choose a pedagogical action.

Additional classifier baselines. A follow-up classifier-family comparison reused the same source-code signal but projected it to 128 SVD components so that all four shallow classifier families could train on the same representation. On that compressed space, linear SVM achieved the best macro F_1 at 0.383, followed by SGD at 0.378, Logistic Regression at 0.365, and Random Forest at 0.358. All four compressed-representation models remain below the main full-sparse source-code model (0.426 macro F_1), which suggests that preserving the high-dimensional character n-gram space matters more here than switching among these classifier families. For deployment, this supports keeping the sparse linear model used by the FastAPI service.

Statistical validation. The held-out test partition contains 42,662 submissions, providing reasonably precise accuracy estimates. The corresponding 95% Wilson confidence intervals are 0.2674–0.2759 for the majority baseline, 0.3297–0.3387 for context metrics, 0.4004–0.4098 for source code, 0.3992–0.4085 for source + context, and 0.7769–0.7847 for code + run. We also repeated the grouped evaluation with seeds 42, 52, and 62 to check whether the model ranking was caused by a particularly favorable split. The same general ordering was observed in all three runs. The majority baseline achieved macro $F_1 = 0.087 \pm 0.001$, followed by context metrics with 0.302 ± 0.025, source code with 0.420 ± 0.005, source + context with 0.399 ± 0.011, and code + run with 0.808 ± 0.007. These results suggest that the relative performance of the models remains stable across the tested unseen-problem splits rather than depending on a single favorable partition.

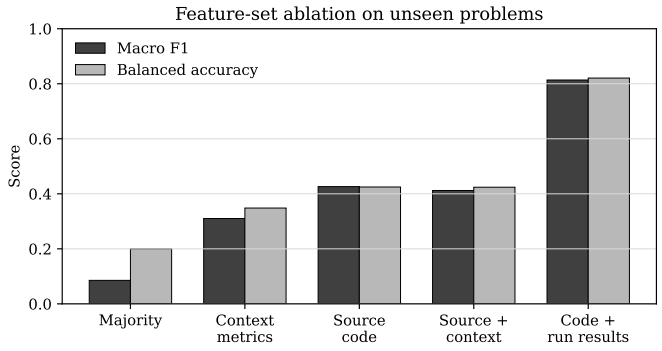


Fig. 2. Feature-set ablation. Source code improves over context metrics, while run results substantially improve the decision layer.

TABLE V
PER-CLASS F_1 FOR TWO MAIN MODEL VARIANTS

Feedback state	Source code	Code + run
Accepted	0.3803	0.9157
Semantic debug	0.4116	0.6099
Execution safety	0.3501	0.6114
Efficiency review	0.4398	0.9712
Syntax repair	0.5490	0.9602

Split-first follow-up. The main balanced benchmark applies class capping before the train-test split, so we also performed an additional experiment with a cleaner sequence. In this setup, the data is first split by problem, the class cap is applied only to the training partition, and the test partition remains unchanged. The resulting test set contains 118,310 submissions and therefore follows a more natural class distribution. The overall ordering remains the same: majority macro $F_1 = 0.139$, context metrics 0.224, source + context 0.319, source code 0.349, and code + run 0.798. The absolute macro-F_1 values are lower because accepted and semantic-debug submissions dominate the uncapped test set, but the relative ordering of the different evidence sources does not change.

The per-class scores in Table V are more useful than looking only at the overall metrics. The source-code model performs best on syntax repair and efficiency review, where surface structure often reveals the issue directly, but it is weaker on accepted and semantic-debug states because two visually similar solutions can still differ on hidden tests.

RQ3 also concerns how SocratesAI should handle low-confidence predictions. On held-out source-code predictions, requiring confidence of at least 0.35 retains 58.1% of submissions and raises macro F_1 to 0.493. Raising the threshold to 0.50 retains 11.5% of submissions with macro $F_1 = 0.608$, and raising it to 0.70 retains 2.9% with macro $F_1 = 0.748$. The current prototype therefore uses 0.35 as a pragmatic operating point: it improves reliability over unrestricted source-only prediction without collapsing coverage to a tiny minority of cases.

Figure 3 shows how the source-code model confuses different feedback states. Accepted submissions are often confused

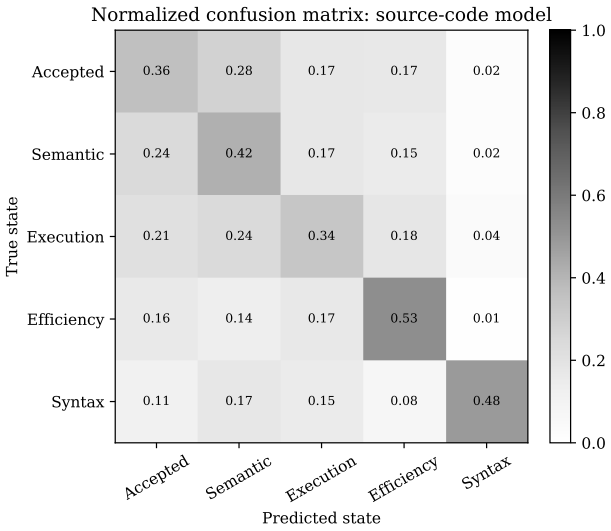


Fig. 3. Error-classification matrix for the source-code model. Off-diagonal cells show which feedback states are confused.

with semantic-debug cases, and execution-safety problems are spread across several predicted states. This is exactly why the source-only model should be treated as an early estimate rather than a final diagnosis.

V. INTEGRATION INTO SOCRATESAI

The trained model is placed between code analysis and feedback generation layers in SocratesAI. In the current prototype, a Java 21 Spring Boot backend collects analyzer output, session context, and the student's source code, serializes those fields as JSON, and calls the FastAPI endpoint `/predict-mentor-state`. The Python ML service reconstructs the same source metrics used during training, applies the fitted pipeline, and returns a response that includes the predicted mentor state, confidence score, per-class scores, and a mapped feedback action.

The Spring Boot backend uses that action for bounded feedback generation and stores the returned mentor state and confidence in `interaction_logs`. When ML mode is enabled but confidence is below the current operating threshold of 0.35, the backend keeps the ML prediction for logging but falls back to the rule selector for the final action. This design keeps the prediction connected to the product without allowing an uncertain prediction to drive the intervention.

The implementation has two modes: source-only mode, in which the mentor can make an early estimate while the student is still editing, and code-and-run mode, in which test execution evidence is also available and the prediction becomes much more reliable. This aligns with a realistic CS1 workflow: a student writes code, runs or submits it, and receives a short intervention tied to the observed failure type.

The main purpose of this trained model is to select the pedagogical direction instead of providing the final answer. For example, a high-confidence efficiency-review prediction can lead to a prompt about time complexity, while

a low-confidence semantic-debug prediction can trigger a more cautious leading question about expected versus actual output. In the current prototype, LLMs are used only to phrase the selected action in natural language. The prompt is bounded so that it does not provide a full solution, rewrite the student's program, or ignore the predicted feedback state. The repository also stores post-hoc outcomes such as `feedback_helpful`, `feedback_rating`, `resolved_after_feedback`, and expert labels, which makes later classroom or expert validation possible even though those data are not the main evaluation source in the present paper.

VI. Discussion and Limitations

The results show that a machine learning model can learn meaningful feedback-state signals from real programming submissions, but source code alone is not enough for reliable tutoring on unseen problems. Run results change the problem substantially. This is not a weakness of the experiment; it is a design lesson for SocratesAI. A mentor that ignores passed tests, time, and memory discards evidence that helps separate failure modes.

Several limitations bound this conclusion. First, the labels are proxy mentor states derived from Codeforces verdicts, not teacher-written pedagogical annotations. Second, the code-and-run benchmark should be interpreted as a post-execution upper bound, not as evidence that ML is needed once a platform already exposes the final verdict token directly. Third, the main ablation table uses a balanced benchmark, although both the additional three-split rerun and the split-first follow-up preserve the same ordering across unseen-problem partitions. Fourth, although SocratesAI now logs mentor-state predictions, usefulness ratings, resolution flags, and expert labels, those interaction logs are not yet large enough to serve as the main quantitative evaluation dataset. Finally, beginner-rated Codeforces problems differ from real CS1 tasks that may also emphasize style, decomposition, and explanation quality.

These limitations are acceptable only within the scope claimed here: the paper presents a feasible ML decision layer for a virtual mentor, not a finished educational product and not evidence of learning gains. The next research step is to keep collecting SocratesAI interaction logs and validate the model on target-domain student data.

VII. Conclusion

This paper presents the feedback-state decision layer of a virtual programming mentor as a supervised machine learning task. Using real Python submissions from beginner-level Codeforces problems, the study shows that source-code features provide useful early signals, but they are still not enough when the model is tested on unseen problems. On the main balanced benchmark, the source-code model achieves macro $F_1 = 0.426$, while the code-and-run model achieves macro $F_1 = 0.814$. A split-first follow-up with an uncapped test partition preserves the same ordering, although with lower absolute macro-F_1 values under the natural class skew.

The main supported conclusion is pragmatic: a virtual mentor should not rely only on generated explanations or on the submitted source code, but should estimate an appropriate feedback state, use execution results when they are available, and store predictions and outcomes inside the application for later validation. In SocratesAI, this connects the external Codeforces study to the deployed prototype: the ML service predicts the feedback state, the feedback layer turns it into a bounded hint, and `interaction_logs` form the basis for future classroom evaluation. Stronger claims about educational effectiveness still require real classroom evidence.

Acknowledgment

The authors thank the course instructors and reviewers whose feedback helped narrow the project from a general virtual mentor prototype to a focused machine learning study.

References

[1] H. Keuning, J. T. Jeuring, and B. Heeren, "A systematic literature review of automated feedback generation for programming exercises," *ACM Transactions on Computing Education*, vol. 19, no. 1, Article 3, 2019.

[2] M. Messer, N. C. C. Brown, M. Kölling, and M. Shi, "Automated grading and feedback tools for programming education: A systematic review," *ACM Transactions on Computing Education*, vol. 24, no. 1, Article 10, 2024.

[3] S. S. Kumar, M. A. Lones, M. Maarek, and H. Zantout, "Navigating the landscape of automated feedback generation techniques for programming exercises," *ACM Transactions on Computing Education*, vol. 25, no. 4, Article 54, 2025.

[4] P. Denny, S. MacNeil, J. Savelka, L. Porter, and A. Luxton-Reilly, "Desirable characteristics for AI teaching assistants in programming education," in *Proc. ITiCSE*, 2024, pp. 408–414.

[5] U. Z. Ahmed, S. Sahai, B. Leong, and A. Karkare, "Feasibility study of augmenting teaching assistants with AI for CS1 programming feedback," in *Proc. ACM SIGCSE TS*, 2025, pp. 11–17.

[6] M. Kazemitabaar *et al.*, "CodeAid: Evaluating a classroom deployment of an LLM-based programming assistant that balances student and educator needs," in *Proc. CHI*, 2024, Article 650.

[7] P. Bassner, E. Frankford, and S. Krusche, "Iris: An AI-driven virtual tutor for computer science education," in *Proc. ITiCSE*, 2024, pp. 394–400.

[8] R. Liu, J. Zhao, B. Xu, C. Perez, Y. Zhukovets, and D. J. Malan, "Improving AI in CS50: Leveraging human feedback for better learning," in *Proc. ACM SIGCSE TS*, 2025, pp. 715–721.

[9] B. Sheese, M. Liffiton, J. Savelka, and P. Denny, "Patterns of student help-seeking when using a large language model-powered programming assistant," in *Proc. ACE*, 2024, pp. 49–57.

[10] S. Marwan, B. Akram, T. Barnes, and T. W. Price, "Adaptive immediate feedback for block-based programming: Design and evaluation," *IEEE Transactions on Learning Technologies*, vol. 15, no. 3, pp. 406–420, 2022.

[11] B. A. Becker, G. Glanville, R. Iwashima, C. McDonnell, K. Goslin, and C. Mooney, "Effective compiler error message enhancement for novice programming students," *Computer Science Education*, vol. 26, no. 2–3, pp. 148–175, 2016.

[12] A. Lobanov, T. Bryksin, and A. Shpilman, "Automatic classification of error types in solutions to programming assignments at online learning platform," in *Artificial Intelligence in Education*, 2019, pp. 174–178.

[13] K. Wang, R. Singh, and Z. Su, "Search, align, and repair: Data-driven feedback generation for introductory programming exercises," in *Proc. PLDI*, 2018, pp. 481–495.

[14] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[15] MatrixStudio, "Codeforces-Python-Submissions," Hugging Face dataset repository, accessed May 30, 2026. [Online]. Available: https://huggingface.co/datasets/MatrixStudio/Codeforces-Python-Submissions